

MACROFIT AI

Resumen del sexto bloque de trabajo

DIA 6 | Registro real de usuarios y seguridad de contraseñas

Autor	Matias Gonzalez
Rama	feature/user-registration
Commit	aa6fe57 - feat: implement user registration
Estado final	Rama sincronizada y working tree clean

Objetivo del documento

Registrar la implementacion del primer flujo real de negocio de MacroFit AI: alta de usuarios, validacion de entrada, proteccion de contraseñas, persistencia en PostgreSQL, respuesta publica segura y control de correos duplicados.

1. Trabajo realizado

El backend dejo de utilizar un registro simulado y paso a crear usuarios reales en PostgreSQL. La funcionalidad se implemento respetando la arquitectura por capas del proyecto.

- Integracion de feature/backend-setup en develop.
- Creacion de la rama feature/user-registration.
- Ampliacion del schema RegisterRequest con nombre, correo y contraseña.
- Creacion de UserResponse para devolver solo campos publicos.
- Implementacion de create_user() en el repository.
- Creacion del modulo app/core/security.py.
- Proteccion de contraseñas con Argon2id mediante pwdlib.
- Implementacion del service de registro y control de correo duplicado.
- Conexion del endpoint con Depends(get_db).
- Pruebas completas desde Swagger y PostgreSQL.

Flujo implementado

POST /auth/register -> Pydantic -> API -> Service -> Repository -> PostgreSQL -> UserResponse

2. Responsabilidad de cada capa

Capa / archivo	Responsabilidad
api/auth.py	Recibe POST /auth/register, obtiene la sesion, llama al service y declara la respuesta 201.
schemas/auth.py	Valida nombre, correo y contraseña; UserResponse filtra campos sensibles.
services/auth_service.py	Comprueba duplicados, genera el hash y coordina la creacion.
repositories/user_repository.py	Consulta por correo y realiza la insercion mediante SQLAlchemy.
core/security.py	Genera hashes Argon2id con pwplib.
models/user.py	Representa la tabla users y sus restricciones.
PostgreSQL	Almacena el usuario y el hash, nunca la contraseña original.

Schemas utilizados

- RegisterRequest: name entre 2 y 100 caracteres, email valido y password minimo de 8 caracteres.
- UserResponse: id, name, email, preferred_language, is_active y created_at.
- password y password_hash quedan excluidos de la respuesta publica.

3. Seguridad y pruebas

La contraseña original se transforma antes de alcanzar el repository. Argon2id incorpora un salt aleatorio, por lo que la misma contraseña genera hashes diferentes y no puede recuperarse directamente desde el valor almacenado.

Escenario	Resultado	Significado
Correo nuevo	201 Created	Usuario creado correctamente.
Correo duplicado	409 Conflict	El service detiene el registro.
Datos invalidos	422 Validation Error	Pydantic rechaza la entrada.

- Se registro el usuario de prueba desde Swagger.
- Se repitio el registro y se obtuvo 409 Conflict.
- La respuesta 201 no incluyo campos sensibles.
- Se consulto PostgreSQL directamente y password_hash comenzaba por \$argon2id\$.
- El idioma predeterminado se guardo como es y la cuenta como activa.

Respuesta publica comprobada

```
{ id, name, email, preferred_language, is_active, created_at }
```

4. Errores encontrados y soluciones

Problema	Aprendizaje / solucion
Parametros y valores confundidos	En definiciones se usa nombre: tipo; al llamar funciones se usa nombre=valor.
Resultado del hash descartado	Se debe devolver password_hasher.hash(password), no la contraseña original.
Codigo despues de return	Todo lo situado despues de return no se ejecuta en esa llamada.
Import backend.app incorrecto	Ejecutando desde backend, el paquete raiz es app.
SQLAlchemy no encontraba Profile	Se registraron User y Profile desde app/models/__init__.py.
Espacios al final del archivo	git diff --check permitio detectar y limpiar trailing whitespace.

Conceptos consolidados

- API recibe HTTP; service aplica reglas; repository accede a datos.
- Pydantic valida entrada y controla salida.
- El hash no es cifrado reversible.
- Depends(get_db) proporciona y cierra la sesion de SQLAlchemy.
- response_model evita exponer password_hash.
- Los codigos HTTP describen el resultado de la operacion.

5. Estado final y proximos pasos

- Registro real operativo y probado.
- Contraseñas protegidas con Argon2id.
- Correos duplicados bloqueados.
- Rama feature/user-registration subida a GitHub.
- Commit aa6fe57 creado correctamente.
- Working tree limpio al finalizar.

Proximos pasos

- Actualizar el backlog con las tareas completadas del registro.
- Integrar la rama en develop tras cerrar la documentacion.
- Implementar verify_password() para comprobar contraseñas.
- Crear POST /auth/login.
- Generar tokens JWT con expiracion.
- Añadir pruebas automaticas de registro y autenticacion.
- Implementar el flujo de completar perfil.

HITO: MacroFit AI ya puede registrar usuarios reales de forma segura y persistente.